EF Core məlumat yükləmə növləri

EF Core-da əlaqəli məlumatı nə vaxt və necə yüklədiyin həm nəticənin düzgünlüyünü, həm sorğu sayını, həm də yaddaş sərfini dəyişir. Eager, explicit və lazy loading üç yükləmə üsuludur. Tracking isə ayrıca qərardır: yüklənmiş obyektin dəyişiklikləri izlənilsinmi? Bu iki qərarı qarışdırmamaq dərsin əsas məqsədidir.

İlkin materiallar `ef_core_tracking.pdf`, `files/04_ef_core.pdf` və `files/01_dotnet_core_fundamentals.pdf` idi. Explicit və lazy loading şərtləri, filtered Include, split query kompromisləri və yanlış performans ümumiləşdirmələri burada tamamlanır.

Mündəricat

EF Core məlumat yükləmə növləri **1**

1 Modeli başa düşək 2

2 Sorğu nə vaxt işləyir 2

3 Eager loading əvvəlcədən əlaqələri seçir 2

4 Explicit loading sonradan açıq əmr verir 3

5 Lazy loading xassəyə çatanda gizli yükləyir 3

6 N+1 problemini sayla başa düşək 4

7 Projection daha az məlumat seçir 4

8 Single query split query və çoxalma 5

9 Tracking yükləmə növü deyil 5

10 Seçim və müsahibə məşqi 6

11 Rəsmi mənbələr 6

1 Modeli başa düşək

ORM, Object Relational Mapper, proqram obyektləri ilə əlaqəli verilənlər bazası cədvəlləri arasında uyğunluq quran vasitədir. EF Core .NET üçün ORM-dir. **Entity** bazada saxlanan məlumatı təmsil edən obyektdir. **Primary key**, əsas açar, sətri fərqləndirir. **Foreign key**, xarici açar, başqa cədvəlin açarına istinaddır. **Navigation property** obyekt modelində həmin əlaqə ilə başqa obyektə və ya obyektlər toplusuna keçid verir.

Bir `Order` sifarişinin bir `Customer` müştərisi və çoxlu `OrderItem` sətri olsun. `CustomerId` xarici açardır. `Customer` reference navigation, yəni bir obyektə keçiddir. `Items` collection navigation, yəni çoxlu obyektə keçiddir. İki cədvəlin şərtə görə birləşdirilməsi SQL-də **JOIN** adlanır.

C#

```
public class Order
{
    public int Id { get; set; }
    public int CustomerId { get; set; }
    public Customer Customer { get; set; } = null!;
    public List<OrderItem> Items { get; set; } = new();
}
```

Bu model fraqmentidir; `Customer`, `OrderItem` və DbContext ayrıca müəyyən edilir. `int` tam ədəd, `List<T>` dəyişə bilən siyahıdır. `get/set` xassənin oxunması və dəyişdirilməsidir. `null!` başlanğıcda boş istinada görə kompilyator xəbərdarlığını susdurur; runtime-da həqiqətən null olmayacağına zəmanət yaratmır. Boş `Items` siyahısı da əlaqənin bazadan yükləndiyini sübut etmir.

2 Sorğu nə vaxt işləyir

LINQ C#-da məlumat sorğularını ifadə edən metodlar və dil imkanlarıdır. `Where` filtr, `OrderBy` sıralama, `Select` seçilən nəticə formasını qurur. `IQueryable<T>` sorğunu provider-in tərcümə edə bildiyi ifadə kimi saxlayır. Provider burada C# ifadəsini SQL-ə çevirən EF Core hissəsidir. **Deferred execution** sorğunun qurulduğu anda deyil, nəticə istənəndə icrasıdır.

C#

```
var query = db.Orders.Where(o => o.CustomerId == 7);
// Hele netice materialize olunmayib.
var list = await query.ToListAsync(ct);
// SQL burada gonderilir ve obyektler yaranir.
```

Materialization bazadan gələn sətirlərin C# obyektlərinə çevrilməsidir. `ToListAsync` nəticəni siyahıya yığır. `await` asinxron əməliyyatın bitməsini gözləyir. `ct` ləğv siqnalını daşıyan `CancellationToken`-dir. `IEnumerable<T>` sadəcə elementləri gəzmək müqaviləsidir; “həmişə bütün məlumatı əvvəl RAM-a yükləyir” tərifi yanlışdır. `AsEnumerable()` özü sorğunu icra etmir, ancaq sonrakı LINQ to Objects əməliyyatlarını C# tərəfində tətbiq edir. `ToList()` isə nəticəni yaddaşa yığır.

Sorğu obyektini DbContext bağlandıqdan sonra icra etmək olmaz. Context sorğunu göndərən və lazım gəlsə izləmə aparan iş vahididir. Onun ömrü nəticənin alınmasına çatmalıdır.

3 Eager loading əvvəlcədən əlaqələri seçir

Eager loading əsas nəticə alınanda lazım olan əlaqələri də sorğuya daxil etməkdir. `Include` bir səviyyəni, `ThenInclude` həmin əlaqənin davamını göstərir.

C#

```
var orders = await db.Orders
    .Include(o => o.Customer)
    .Include(o => o.Items)
        .ThenInclude(i => i.Product)
    .Where(o => o.CustomerId == 7)
    .ToListAsync(ct);
```

Bu nümunədə sifarişlər, onların müştərisi, sətirləri və həmin sətirlərin məhsulları tələb olunur. `Product` məhsul əlaqəsidir. Sorğu hansı əlaqələrin lazım olduğunu açıq göstərir. Ancaq eager loading həmişə bir SQL əmri demək deyil: single query rejimində adətən JOIN-lər, split query rejimində əlavə sorğular yarana bilər.

Bir müştərinin adı hər sifariş sətrində təkrar ötürülə bilər. Böyük mətn və ikili məlumat, məsələn şəkil, təkrarlanırsa şəbəkə yükü artır. “Əlaqə lazımdırsa bütün entity-ni Include et” əvəzinə nəticədə hansı sahələrin lazım olduğunu müəyyənləşdir.

4 Explicit loading sonradan açıq əmr verir

Explicit loading əsas entity alındıqdan sonra əlaqəni ayrıca əmrlə yükləməkdir. Seçim metodda aydın görünür. Məsələn, istifadəçi “sifariş detalları” düyməsini açdıqda sətirləri gətirə bilərsən.

C#

```
var order = await db.Orders
    .SingleAsync(o => o.Id == 42, ct);

await db.Entry(order)
    .Collection(o => o.Items)
    .LoadAsync(ct);

await db.Entry(order)
    .Reference(o => o.Customer)
    .LoadAsync(ct);
```

`SingleAsync` dəqiq bir nəticə gözləyir; sıfır və ya birdən çox nəticə xətadır. `Entry` entity barədə EF Core məlumatına çıxış verir. `Collection` çoxlu, `Reference` tək əlaqəni seçir. `LoadAsync` seçilmiş əlaqəni yükləyir. Tədris nümunəsi tracking aktiv olan context nəzərdə tutur; müxtəlif EF Core versiyalarında detached entity davranışını ayrıca yoxlamaq lazımdır.

Yalnız say lazımdırsa bütün sətirləri yükləmə:

C#

```
var count = await db.Entry(order)
    .Collection(o => o.Items)
    .Query()
    .CountAsync(ct);
```

`Query` əlaqəni IQueryable kimi verir, `CountAsync` bazada say hesablayır. Bu əməliyyat `Items` kolleksiyasının tam yüklənməsi demək deyil. Explicit loading nəzarəti artırır, amma dövrdə hər entity üçün çağırılsa yenə N+1 problemi yarada bilər.

5 Lazy loading xassəyə çatanda gizli yükləyir

Lazy loading əlaqəyə ilk müraciət zamanı onun lazım gəlsə avtomatik alınmasıdır. `order.Customer.Name` sətri görünüşcə yaddaş oxunuşudur, amma bazaya sorğu göndərə bilər. EF Core-da bu davranış adi modeldə avtomatik aktiv deyil.

Proxy üsulunda `Microsoft.EntityFrameworkCore.Proxies` paketi və `UseLazyLoadingProxies()` konfiqurasiyası istifadə edilir. **Proxy** davranışı tutmaq üçün yaradılmış əlavə obyekt tipidir. Sinifdən miras almaq mümkün olmalı, uyğun navigation xassələri `virtual` olmalıdır. `virtual` davranışın törəmə sinifdə əvəz edilməsinə imkan verir. Digər yanaşma `ILazyLoader` xidmətindən istifadədir; proxy tələbləri həmin üsul üçün eyni deyil.

C#

```
// Proxy usulunda modelden fraqment:
public virtual Customer Customer { get; set; } = null!;

// Konfiqurasiya fraqmenti:
options.UseLazyLoadingProxies().UseSqlServer(connection);
```

`connection` baza bağlantı parametridir. `UseSqlServer` SQL Server provider-ini seçir. Lazy loading üçün context-in işlək olması lazımdır. Cavab JSON-a çevrilərkən serializer navigation-ları oxuyub əlavə SQL sorğuları və ya dairəvi obyekt istinadları problemi yarada bilər. DTO qaytarmaq və lazım olan məlumatı açıq seçmək API-də daha nəzarətli yanaşmadır.

6 N+1 problemini sayla başa düşək

N+1 bir əsas sorğudan sonra N obyekt üçün ayrıca sorğu göndərilməsidir. 50 sifariş alıb hər birinin hələ yüklənməmiş sətirlərinə lazy keçsən, sadə ssenaridə 1 + 50 = 51 sorğu alınar. Hər şəbəkə gediş gəlişi 8 ms olsa, təkcə 50 əlavə gediş gəliş təxminən 400 ms gözləmə yaradar. Bu hesabi nümunədir, ölçülmüş benchmark deyil.

Eyni müştəri bir neçə sifarişdə təkrar istifadə olunursa tracking və əvvəl yüklənmiş əlaqələr faktiki sayın dəyişməsinə səbəb ola bilər. Buna görə “100 sifariş həmişə 101 sorğudur” hökmü yox, konkret yüklənmə vəziyyəti ilə izah edilməlidir. **Roundtrip** tətbiqdən bazaya gedib cavabın geri gəlməsi dövrüdür.

Həll olaraq eager loading, projection və ya toplu əlavə sorğu seçilə bilər. **Batch** bir çox obyekt üçün işi toplu görməkdir. `foreach` içində explicit `LoadAsync` yazmaq gizliliyi aradan qaldırır, amma sorğu sayını avtomatik azaltmır. SQL əmrlərinin logunu açıb real sayı ölçmək lazımdır.

7 Projection daha az məlumat seçir

Projection nəticəni lazım olan sahələrə çevirməkdir. DTO read-only, yəni yalnız oxunan ekran üçün tam entity qrafından daha uyğun ola bilər.

C#

```
var rows = await db.Orders
    .Where(o => o.CustomerId == 7)
    .Select(o => new
    {
        o.Id,
        CustomerName = o.Customer.Name,
        ItemCount = o.Items.Count
    })
    .ToListAsync(ct);
```

Burada `new { ... }` adsız nəticə tipi yaradır. EF Core uyğun ifadələri SQL-ə çevirə bilirsə müştəri adı və say bazada hesablanır; əvvəlcə bütün `Items` obyektlərini çəkmək lazım deyil. Bu projection üçün Include tələb olunmur. Lakin hər C# metodunun SQL qarşılığı yoxdur; tərcümə dəstəyi provider və versiyadan asılıdır. Entity obyekti daxil etməyən saf scalar projection-da tracker-ə entity əlavə olunmur. Scalar tək dəyərdir,

məsələn ad və ya say.

8 Single query split query və çoxalma

Bir sifarişin 10 item-i və ayrıca 4 payment-i varsa eyni səviyyədə iki kolleksiyanın JOIN-i 10 x 4 = 40 nəticə sətri yarada bilər. 100 belə sifariş üçün 4000 sətir ötürülə bilər. **Cartesian explosion** bir-birindən müstəqil kolleksiyaların birləşməsi ilə nəticə sətirlərinin çoxalmasıdır. Nested, yəni biri digərinin içində olan əlaqəni, sibling, yəni eyni səviyyəli əlaqə ilə qarışdırma.

C#

```
var orders = await db.Orders
    .Include(o => o.Items)
    .Include(o => o.Payments)
    .AsSplitQuery()
    .ToListAsync(ct);
```

`AsSplitQuery` uyğun kolleksiya yükləməsini bir neçə SQL sorğusuna bölür. Dəqiq sorğu sayını “hər Include üçün bir” kimi əzbərləmək olmaz; reference əlaqələr, sorğu forması, provider və versiya təsir edir. Nəticənin çoxalması azalır, amma əlavə roundtrip və buffering, yəni aralıq nəticələri yaddaşda saxlama, xərci yarana bilər.

Birinci və ikinci SQL arasında başqa transaction məlumatı dəyişərsə bir-birinə tam uyğun olmayan anların məlumatı gələ bilər. Transaction bir neçə baza əməliyyatının müəyyən bütövlük qaydaları altında birləşdirilməsidir. Uyğun isolation, yəni paralel işlər arasında görünmə qaydası, lazım ola bilər və onun da xərci var. Səhifələmədə `OrderBy`-a unikal əlavə açar, məsələn `ThenBy(o => o.Id)`, vermək eyni tarixli sətirlərin qeyri-müəyyən sırasını aradan qaldırır.

9 Tracking yükləmə növü deyil

Change tracker yüklənmiş entity-lərin vəziyyətini və dəyişikliklərini izləyir. **Snapshot** ilkin dəyərlərin yadda saxlanan surətidir. **Identity resolution** eyni açarlı entity-nin eyni nəticə qrafında bir nüsxə kimi birləşdirilməsidir. **Navigation fixup** əvvəl yüklənmiş uyğun entity-lərin əlaqə xassələrinin avtomatik uyğunlaşdırılmasıdır.

`AsNoTracking` əsas context tracker-inə həmin sorğu entity-lərini əlavə etmir. Oxuma ssenarilərində faydalıdır, amma “həmişə 20-30 faiz sürətlidir” deyə sabit rəqəm yoxdur. Tracking əvvəl yüklənmiş entity-ni təkrar istifadə etməklə bəzi hallarda xərci azalda bilər. `AsNoTrackingWithIdentityResolution` isə əsas tracker-dən ayrı, nəticə daxilində təkrar nüsxələri birləşdirir.

Tracking ilə alınan entity dəyişdirilib `SaveChangesAsync` çağırılanda yenilənə bilər. No-tracking entity-ni dəyişmək təkbaşına UPDATE göndərmir. `Update(entity)` bütün dəyişdirilə bilən sahələri modified işarələyə bilər; disconnected, yəni izləmə mühitindən ayrılmış məlumat üçün yalnız icazəli sahələri attach və ya izlənən entity üzərinə köçürmək daha dəqiq ola bilər. `Attach` entity-ni tracker-ə tanıdır.

Filtered Include əlaqənin müəyyən hissəsini yükləyir: məsələn `Include(o => o.Items.Where(i => i.Price > 10))`. Əvvəl eyni tracking context-də başqa item-lar yüklənibsə fixup səbəbindən nəticə gözlədiyindən geniş görünə bilər. Üstəlik əlaqə loaded işarələnə bilər, halbuki onun yalnız bir hissəsi alınıb. Təmiz yeni context və ya məqsədə uyğun no-tracking sorğusu bu vəziyyəti idarə etməyə kömək edir.

10 Seçim və müsahibə məşqi

Ehtiyac	Başlanğıc seçim və səbəb
Siyahıda ad və say	Projection. Lazımsız entity və sütunları götürmür.
Əlaqələr əvvəlcədən bəllidir	Eager loading. Məlumat ehtiyacını bir yerdə açıq göstərir.
Bir obyektin detalları sonradan lazımdır	Explicit loading. Əlavə SQL çağırışı nəzarətdədir.
Navigation-a rahat keçid tələb olunur	Lazy loading mümkündür; gizli sorğu sayı və context ömrü ölçülməlidir.
Bir neçə böyük kolleksiya var	Single və split query-ni real məlumatla müqayisə et.

DbContext-i Task.WhenAll ilə paylaşa bilərəmmi? Eyni context-də eyni anda iki baza əməliyyatı işlətmə. Ardıcıl await et və ya həqiqətən müstəqil işlər üçün ayrıca context yarat. İki context yaratmaq onların avtomatik eyni transaction-da olması demək deyil.

Məşq. 20 sifariş üçün item sayını göstərən ekran qur. Əvvəl lazy navigation-a keç, sonra eager loading, sonra scalar projection yaz. Hər variantın SQL sayını, qayıdan sütunlarını və vaxtını qeyd et. Gözlənən nəticə: lazy variantda çoxsaylı əlavə sorğu riski, eager variantda tam item məlumatı, projection-da isə yalnız lazım olan say. Dəqiq vaxt rəqəmini ölçmədən elan etmə.

11 Rəsmi mənbələr

- Microsoft Learn - Loading related data
- Microsoft Learn - Eager loading
- Microsoft Learn - Explicit loading
- Microsoft Learn - Lazy loading
- Microsoft Learn - Single versus split queries
- Microsoft Learn - DbContext configuration

Yoxlama tarixi 28 oktyabr 2026. C# fraqmentləri EF Core relational provider-i nəzərdə tutur; yaradılan SQL-i konkret layihədə log və `ToQueryString()` ilə yoxlamaq lazımdır. `ToQueryString` sorğunun SQL təsvirini göstərir; bütün son icra əmrlərini görmək üçün log daha uyğundur.